



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 09

NOMBRE COMPLETO: Tavera Castillo David Emmanuel

N° de Cuenta: 320054831

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 10/30/2025

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Durante la practica se nos piden 3 ejercicios obligatorios y 1 extra, el primer ejercicio consta de separar el modelo del arco propuesto en el cuestionario previo en diferentes modelos para poder agregar animaciones para lo cual utilizaremos Blender.



El segundo ejercicio nos pide crear una animación donde se muestre la palabra PROYECTO CGEIHHC desplazándose de derecha a izquierda como un letrero LED de forma cíclica.

Para esto debemos de crear primero la textura de la frase, hay dos formas de hacerlo, llamar letra por letra y formar la animación, o juntar en una sola imagen/textura la frase y solo desplazarla. Optamos por la segunda opción utilizando la siguiente imagen.

PROYECTO CGEIHHC

La lógica de código para programar la animación es la siguiente

```
//Frase
toffsetfraseu += 0.001;
toffsetfrasev = 0.0;
if (toffsetfraseu > 1.0)
    toffsetfraseu = 0.0;
toffset = glm::vec2(toffsetfraseu, toffsetfrasev);
model = modelaux;
model = glm::translate(model, glm::vec3(-0.145f, 0.05f, 0.59f));
model = glm::scale(model, glm::vec3(2.9f, 1.8f, 2.0f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));

glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1.0f, 0.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
Frase.UseTexture();
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
meshList[4]->RenderMesh();
```

Los resultados se mostrarán en el video adjunto.

El ultimo ejercicio obligatorio nos pide que al presionar una tecla las puertas del arco se abran, para esto se debe implementar dos animaciones distintas, la primera consta de rotar la puerta, la segunda tendrá una animación similar a una puerta deslizante sin que atraviese el arco.

El siguiente código muestra la lógica de la puerta derecha siendo esta la que tiene una animación de rotación.

```
//PuertaD
model = modelaux;
model = glm::translate(model, glm::vec3(1.822f, -3.35f, 0.0f));
if (mainWindow.getAnimacion() > 0.5f) {
    if (rota < 90.0f) {
        rota += 2.0f * deltaTime;
    }
}
else {
    if (rota > 0.0f) {
        rota -= 2.0f * deltaTime;
    }
}
model = glm::rotate(model, rota * toRadians, glm::vec3(0.0f, -1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
PuertaD.RenderModel();
```

El siguiente código muestra la lógica de la puerta izquierda la cual tiene la animación de desplazarse evitando el arco.

```
//PuertaI
model = modelaux;
model = glm::translate(model, glm::vec3(-1.85f, -3.35f, 0.0f));
if (mainWindow.getAnimacion() > 0.5f) {
    if (tz > -0.65f) {
        if (tz > -0.65f) {
            tz -= 0.05f * deltaTime;
        }
    }
    else {
        if (tx > -1.96f) {
            tx -= 0.05f * deltaTime;
        }
    }
}
else {
    if (tx < 0.0f) {
        tx += 0.05f * deltaTime;
    }
    else {
        if (tz < 0.0f) {
            tz += 0.05f * deltaTime;
        }
    }
}
model = glm::translate(model, glm::vec3(tx, 0.024f, tz));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
PuertaI.RenderModel();
```

En ambos casos tenemos la tecla para activar la animación siendo la Q, además se agrega una tecla extra para que los modelos vuelvan a su posición inicial siendo la letra E.

Los resultados se mostrarán en el video adjunto.

El ejercicio extra nos pide realizar con nuestro dado de 8 caras de prácticas anteriores una animación donde simule caer girando sobre el piso y muestre un numero aleatorio cada vez que se ejecute esta animación, la animación debe de repetirse utilizando una tecla.

La lógica para instanciar la primitiva del dado la podemos ver en la practica 6 por lo que nos ahorraremos esa explicación por el momento.

Para mostrar de manera coherente la construcción de nuestro dado con animación necesitaremos desglosar el siguiente código.

```
UpdateOctahedron(deltaTime);
model = glm::mat4(1.0);
model = glm::translate(model, octaPos);
model = glm::rotate(model, octaRot.x, glm::vec3(1, 0, 0));
model = glm::rotate(model, octaRot.y, glm::vec3(0, 1, 0));
model = glm::rotate(model, octaRot.z, glm::vec3(0, 0, 1));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));

octaTexture.UseTexture();
meshList[7]->RenderMesh();
if (mainWindow.getDado()==true)
{
    RollOctahedron();
}
```

Observamos la primera función *UpdateOctahedron* que recibe como parámetro *deltaTime*, si la animación no se ejecuta la función no retorna nada, en caso contrario asigna las velocidades con la que se ve el rebote y ajusta la posición final dependiendo de la rotación que devuelva la función *AlignOctahedronToNumber*.

```
void UpdateOctahedron(float dt)
{
    if (!isRolling) return;
    octaVel.y -= 0.05f * dt;
    octaPos += octaVel * dt;
    octaRot += octaRotVel * dt;

    if (octaPos.y <= floorY)
    {
        octaPos.y = floorY;
        octaVel.y *= -0.1f;
        octaRotVel *= 0.1f;

        if (fabs(octaVel.y) < 0.5f)
        {
            isRolling = false;
            AlignOctahedronToNumber(currentNumber);
        }
    }
}
```

La siguiente función devuelve la rotación final del dado dependiendo del numero que reciba.

```
void AlignOctahedronToNumber(int number)
{
    switch (number)
    {
        case 1:
            octaRot = glm::vec3(-1.0f, 0.785f, 0.0f);
            break;

        case 2:
            octaRot = glm::vec3(-4.1f, 0.785f, 0.0f);
            break;

        case 3:
            octaRot = glm::vec3(4.1f, -0.785f, 0.0f);
            break;

        case 4:
            octaRot = glm::vec3(1.0f, -0.785f, 0.0f);
            break;

        case 5:
            octaRot = glm::vec3(1.0f, 0.785f, 0.0f);
            break;

        case 6:
            octaRot = glm::vec3(4.1f, 0.785f, 0.0f);
            break;

        case 7:
            octaRot = glm::vec3(-4.1f, -0.785f, 0.0f);
            break;

        case 8:
            octaRot = glm::vec3(-1.0f, -0.785f, 0.0f);
            break;
    }
}
```

La última función controla la animación, aquí se calcula el numero random y se da la impresión de que el dado se esta revolviendo.

```
void RollOctahedron()
{
    isRolling = true;
    octaPos = glm::vec3(-1.5f, 0.0f, -2.0f);
    octaVel = glm::vec3(0.0f, -6.0f, 0.0f);

    octaRotVel = glm::vec3(
        (rand() % 200 - 100) / 50.0f,
        (rand() % 200 - 100) / 50.0f,
        (rand() % 200 - 100) / 50.0f
    );

    currentNumber = 1 + rand() % 8;
}
```

Los resultados se mostrarán en el video adjunto.

2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla.

No se tuvo mayor problema con los ejercicios ya que la mayoría son razonamientos lógicos, exceptuando el ejercicio del dado que manejo una complejidad mayor.

3.- Conclusión:

La animación es uno de los efectos mas inmersivos en la computación gráfica, durante la practica se utilizaron diferentes técnicas para animar cosas ya sea de manera natural o llamadas durante teclas.

4.- Bibliografía en formato APA

Ing. José Roque RG (septiembre 2025) Laboratorio de Computación Grafica e Interacción Humano Computadora. Clase Facultad de Ingeniería UNAM.